MIT 8103 Advanced Database Systems — Portfolio Documentation

Case study: EkoDash Logistics Ltd (simulated last-mile delivery company)

Abdullahi Habiba · 2026/A/MIT/0437 · h.abdullahi0372@miva.edu.ng · MIVA Open University, 2026/2027 First Semester

GitHub repository: https://github.com/UIbyBiba/advanced-database-portfolio

1. Selected organisation

EkoDash Logistics Ltd is a simulated on-demand parcel delivery business in Lagos, Abuja and Port Harcourt. A customer books a pickup in the app, the system assigns a nearby motorbike, tricycle or van rider, and the parcel is tracked until delivery. Payment is by card, transfer, in-app wallet or cash. **Users:** customers, riders, and admin staff (dispatch, finance, support). **Major data:** cities and pricing zones, customers and addresses, riders, orders and their status history, payments, wallets and a ledger, ratings, and live rider locations. **Key operations:** place and price an order, assign the nearest rider, update status, take payment, pay riders, and report revenue and earnings. The same synthetic data set (600,000 orders, 50,000 customers, 1,500 riders) is used in all five activities.

2. Portfolio 1 — Database design and modelling

I designed a 12-table PostgreSQL 16 schema around the **delivery_order** fact, with an ER diagram in crow's-foot notation. Design decisions:

- Addresses are a separate entity, because customers send from more than one place.
- Status changes go into an append-only **history** table, so the audit trail is never overwritten.
- Payment is 1:1 with the order (**UNIQUE order_id**).
- A wallet belongs to exactly one customer or rider (an exclusive-arc **CHECK**), and every balance change is written to a ledger.

Business rules are enforced as constraints: Nigerian phone format, status enumerations, "an assigned order must have a rider", "DELIVERED ⇔ delivered_at is set", and no negative wallet balance. A test script tried to break ten rules and **all ten were rejected**. Normalisation was shown from the dispatchers' original spreadsheet through 1NF (the repeating status list), 2NF and 3NF to **BCNF**. The one apparent redundancy, **fee_amount**, is a deliberate price snapshot: a later fee change must not alter past orders.

3. Portfolio 2 — Query processing and optimisation

Six real business queries were measured with **EXPLAIN (ANALYZE, BUFFERS)**, taking the median of five runs, before and after targeted indexing. Each index matches one access path:

- **Q1** customer order history: composite B-tree.
- **Q2** dispatcher queue: partial index on PENDING orders only (40 kB).
- **Q3** rider earnings: covering index giving an index-only scan with 0 heap fetches.
- **Q4** 7-day revenue report: BRIN index, 32 kB against 13 MB for a B-tree.
- **Q5** case-insensitive e-mail lookup: expression index on lower(email).

Query	Purpose	Before (ms)	After (ms)	Speed-up
Q1	Customer order history	36.876	0.023	1,603×
Q2	Dispatcher pending queue	52.982	0.036	1,472×
Q3	Rider monthly earnings	60.035	0.019	3,160×
Q4	Zone revenue (7 days)	157.303	21.027	7× (lossy BRIN: recheck cost)
Q5	E-mail lookup	12.226	0.011	1,111×
Q6	Rider leaderboard	> 60,000	1.231	rewrite + materialized view

The leaderboard (Q6) shows that indexing is not the only tool. The correlated first draft timed out. A join rewrite took 578 ms. Raising work_mem stopped the hash spilling to disk but saved only about 4%. A materialized view, refreshed concurrently in 378 ms, reduced reads to 1.2 ms.

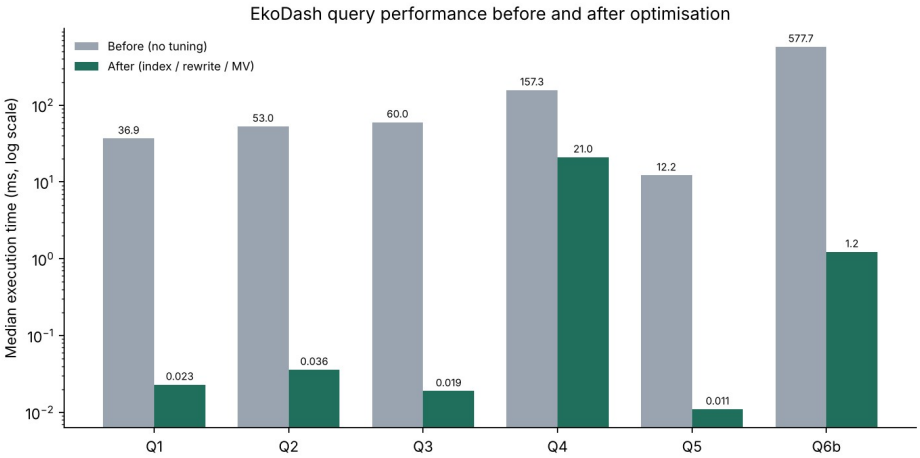


Figure 1. Median execution time before and after optimisation (log scale).

4. Portfolio 3 — Transactions and concurrency

A Python harness opens **two real database sessions** and forces exact interleavings, so each anomaly is reproduced every time:

- **COMMIT and ROLLBACK:** a wallet payment spanning four tables is all-or-nothing. A failed debit also undid the order insert, and a SAVEPOINT discarded only an invalid rating.
- **Lost update:** two payouts left the rider ₦500 short. **SELECT ... FOR UPDATE** fixed it.
- **Race condition:** two riders accepting one job were both told "job is yours". A conditional **UPDATE ... WHERE status = 'PENDING'** and **SKIP LOCKED** fixed it.
- **Non-repeatable read:** it occurred under READ COMMITTED and was prevented by REPEATABLE READ.
- **Write skew:** under REPEATABLE READ, two riders going off duty together left a zone with 0 riders. SERIALIZABLE aborted one transaction with SQLSTATE 40001, and the retry was correctly refused.
- **Deadlock:** PostgreSQL detected it (40P01), and ordered locking prevented it.

output file: portfolio-3-transactions-concurrency/evidence/transaction_demos_output.txt

```
$ python3 scripts/transaction_demos.py D6
D6  WRITE SKEW - rule: a zone must always keep at least 1 rider ON_DUTY (zone 99 has 2)
================================================================================
  [  1.587s] T1  [REPEATABLE_READ] riders on duty in zone 99 = 2
  [  1.588s] T2  [REPEATABLE_READ] riders on duty in zone 99 = 2
  [  1.589s] T2  rule ok (>=2 on duty) -> rider 1500 goes OFF_DUTY
  [  1.589s] T1  rule ok (>=2 on duty) -> rider 1499 goes OFF_DUTY
  [  1.590s] T1  COMMIT ok
  [  1.790s] T2  COMMIT ok
  RESULT: REPEATABLE READ -> {'T1': 'committed', 'T2': 'committed'}; riders left on duty = 0  (business rule BROKEN)

  [  1.810s] T1  [SERIALIZABLE] riders on duty in zone 99 = 2
  [  1.811s] T2  [SERIALIZABLE] riders on duty in zone 99 = 2
  [  1.812s] T1  rule ok (>=2 on duty) -> rider 1499 goes OFF_DUTY
  [  1.812s] T2  rule ok (>=2 on duty) -> rider 1500 goes OFF_DUTY
  [  1.813s] T1  COMMIT ok
  [  2.013s] T2  ERROR 40001 serialization_failure -> ROLLBACK (app should retry)
  RESULT: SERIALIZABLE    -> {'T1': 'committed', 'T2': 'aborted'}; riders left on duty = 1  (rule kept)
  [  2.025s] T2  retry: riders on duty = 1 -> request refused by the rule, nothing written
```

Figure 2. Write skew allowed by REPEATABLE READ, prevented by SERIALIZABLE.

5. Portfolio 4 — NoSQL and advanced data models

The same data was remodelled for the workloads where the relational model is weakest.

- **Document model (MongoDB 7):** 203,899 order documents embed the status timeline, payment, rating and snapshots, so the tracking screen is **one document read** instead of an 8-table join. A $jsonSchema validator rejects bad data. It caught the Portfolio 3 demo orders that had no history on the first migration run.
- **Indexing:** a compound index following the equality-sort rule cut a history query from a COLLSCAN of 203,899 documents to 7. A 2dsphere index with $geoNear finds the nearest on-duty bikes.
- **Key-value model (Redis 8):** live GPS positions (GEOSEARCH in 0.34 ms), ETA cache with TTL, an OTP rate limiter and rider sessions.

Money stays in PostgreSQL because it needs multi-row ACID guarantees. This is polyglot persistence.

6. Portfolio 5 — Distributed and cloud databases

Three real topologies were deployed and tested.

- **PostgreSQL streaming replication:** write visible on the replica in milliseconds; replica refuses writes; 27 MB of peak lag under a 300,000-row burst; sync vs async commit 1.11 ms vs 0.77 ms; failover by promotion in 0.11 s with no committed data lost.
- **MongoDB 3-node replica set:** w:majority vs w:1 latency, reads from secondaries, a multi-document transaction, and SIGKILL of the primary. A new primary was elected after about 10.5 s, and 40/40 application writes succeeded through driver retries.
- **Zone-sharded cluster, shard key {city, _id}:** Lagos on one shard, Abuja and Port Harcourt on another. City queries route to a single shard; queries without the shard key scatter to both. My first attempt left gaps between zone ranges, which made city queries hit both shards. Covering the whole key space with the ranges fixed it.

The cloud design covers placement (API near Lagos, including AWS's Lagos Local Zone; managed databases in a South African region; DR in Europe), Multi-AZ synchronous standby, read replicas, point-in-time recovery, Nigeria Data Protection Act 2023 obligations, and CP/AP choices per data type.

7. Individual technical reflection

Design choices. The biggest choice was to use more than one data model on purpose. I kept every naira movement in PostgreSQL, where constraints and SERIALIZABLE transactions protect it, and moved high-churn tracking and location data to MongoDB and Redis. Writing the business rules as CHECK constraints, rather than leaving them only in application code, meant the database itself proved them in Portfolio 1.

Challenges. My first data generator picked random rows with a correlated sub-query and ran for over ten minutes. Rewriting it as a join brought it to under a minute. Several results also did not match my expectations, and investigating them taught me the most:

- Raising work_mem barely helped Q6, because the cost was the work itself, not the disk spill.
- Q1 kept a small sort step after indexing, because the planner judged that sorting five rows was cheaper.
- Order IDs jumped by 33 after the PostgreSQL failover, because sequences are logged to WAL in batches.
- Gaps between shard-zone ranges silently turned targeted queries into scatter-gather.

Lessons learned. Measure before and after, and read the plan instead of assuming. Isolation levels are a business decision: SERIALIZABLE is only safe if the application retries. Every gain in availability or speed (async replicas, w:1, caching) is paid for in consistency, and that trade-off should be chosen per data type.

8. AI use declaration

Tool: Claude (Anthropic), an AI assistant working in a code-execution workspace. **Used for:** helping to design the case study and schema; writing the SQL, Python and mongosh scripts; running the experiments on PostgreSQL, MongoDB and Redis servers in a cloud sandbox to produce the outputs, plans and screenshots; and drafting the README files and this document. **How I reviewed and verified it:** [TO COMPLETE BEFORE SUBMISSION. Describe honestly what you checked, re-ran and rewrote yourself.]

References

Abadi, D. (2012) 'Consistency tradeoffs in modern distributed database system design', IEEE Computer, 45(2), pp. 37–42.

Brewer, E. (2012) 'CAP twelve years later: how the "rules" have changed', IEEE Computer, 45(2), pp. 23–29.

Connecting Africa (2023) Lagos launches first AWS Local Zone in Africa. https://connectingafrica.com/cloud-networking/lagos-launches-first-aws-local-zone-in-africa

Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. 7th edn. Pearson.

Federal Republic of Nigeria (2023) Nigeria Data Protection Act, 2023.

Kleppmann, M. (2017) Designing Data-Intensive Applications. O'Reilly Media.

MIVA Open University (2026) MIT 8103 Advanced Database Systems [course material].

MongoDB Inc. (2025) MongoDB Manual, version 7.0. https://www.mongodb.com/docs/v7.0/

Ports, D.R.K. and Grittner, K. (2012) 'Serializable Snapshot Isolation in PostgreSQL', PVLDB, 5(12), pp. 1850–1861.

PostgreSQL Global Development Group (2025) PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

Redis Ltd. (2025) Redis documentation. https://redis.io/docs/

Silberschatz, A., Korth, H.F. and Sudarshan, S. (2020) Database System Concepts. 7th edn. McGraw-Hill.